

Caleb Harris

Professor Burge

Software Engineering

October 3, 2026

Code Review Summary – Boggle Solver

For this assignment, I reviewed and improved my original Boggle Solver from Assignment #2 using peer feedback, static code analysis, and unit testing. The review focused on correctness and edge cases, algorithm efficiency, readability and naming, and maintainability. My original solver could search a Boggle grid using recursion and find valid words from a dictionary. However, the review showed me that working code can still have smaller issues that affect its reliability and quality.

During the peer review, my partner identified two important edge cases in my code. First, my `isValid()` method checked whether each grid tile was a string, but an empty string `""` could still pass that check. My partner also pointed out that duplicate words in the dictionary could potentially be added to the solution more than once. These comments helped identify situations that may not occur during normal use but should still be handled by the program.

When reviewing my partner's code, I focused on correctness, efficiency, and readability. I noticed that the solution list could keep previous results if `getSolution()` was called multiple times, so I suggested resetting it before a new search. I also suggested using a set of coordinate tuples such as `(row, col)` instead of a list to track used tiles because sets provide more efficient membership checks. Finally, I recommended organizing the neighboring-tile search logic to make the recursive search easier to read and maintain.

Based on the feedback I received, I made two changes to my solver. I updated `isValid()` so each tile must be a string and contain at least one character. I also updated `getSolution()` to check whether a word was already in `self.solution` before adding it, preventing duplicate results. The solver already used recursion and backtracking to explore neighboring tiles and a visited set to prevent the same tile from being reused in a word. This allowed me to improve the edge cases without changing the main search algorithm.

Static analysis in Codio also identified several PEP 8 style issues, including incorrect blank-line spacing, trailing whitespace, missing spaces after commas, long lines, and a missing newline at the end of the file. The test file also contained indentation issues, unnecessary semicolons, and mixed tabs and spaces. I corrected these issues without changing the intended behavior of the program. After the corrections, both `boggle_solver.py` and `tests.py` passed Codio's static analysis.

After making the improvements, I used regression testing to make sure I had not introduced any new bugs. The final solver passed all **26 instructor-provided tests**, covering different grid sizes, search directions, winding paths, tile reuse, minimum word lengths, grid validation, and special tiles such as Qu and St. I ran the tests multiple times after the final changes, and all 26 continued to pass.

Overall, this assignment showed me that software development involves more than getting a program to produce the correct output. Peer review helped identify edge cases I had not considered, while static analysis showed how code style affects readability and maintainability. Regression testing also showed me the importance of checking existing functionality after making changes. Going through peer review, code improvement, static analysis, and testing gave me a better understanding.